We note that for standalone applications running on power-aware virtualized platforms such as [15], our framework can make use of the OS hints by passing them on to the *Arbitrator*. Hence, our framework and proposed algorithms (Section. 4) can also be used in other power management frameworks [15,23].

2.2 Optimization Formulations

We now formulate the problem of placing N application on M virtualized servers that have power management capabilities. The power-aware application placement problem divides the time horizon in time windows. In each window, we compute the application placement that optimizes the performance-cost tradeoff, i.e., maximizes performance and minimizes cost. The cost metric may consist of management cost, power cost or the application cost incurred due to the migrations that are required to move to the desired placement. We next present various formulations of the application placement problem.

Cost Performance Tradeoff. The generic formulation of the problem solves two sub-problems: (i) application sizing and (ii) application placement. Given a predicted workload for each application, we resize the virtual machine hosting the application and place the virtual machines on physical hosts in a manner such that the cost-performance tradeoff is optimized. In this paper, we focus on the power and migration costs only. Formally, given an old allocation A_o, a performance benefit function $B(A)$, a power cost function $P(A)$, and a migration cost function Mig for any allocation A, we need to find an allocation A_I defined by the variables $x_{i,j}$, where $x_{i,j}$ denotes the resource allocated to application V_i on server S_j, such that the net benefit (defined as the difference between performance benefit and costs) is maximized.

$$maximize \sum_{i=1}^{N} \sum_{j=1}^{M} B(x_{i,j}) - \sum_{j=1}^{M} P(A_I) - Mig(A_o, A_I) \tag{1}$$

Cost Minimization with Performance Constraint. Data centers today are moving towards an SLA-based environment with fixed performance guarantees (e.g., response time of 100 ms with throughput of 100 transactions per second). Hence, in such a scenario, performance is not a metric to be maximized and can be replaced by constraints in the optimization framework. In practice, it amounts to taking away the VM sizing problem away from the *Arbitrator*. The VM sizes are now fixed by the *Performance Manager* based on the SLA and the *Arbitrator* only strives to minimize the overall cost of the allocation. Hence, the optimization problem can now be formulated as

$$minimize \sum_{j=1}^{M} P(A_I) + Mig(A_o, A_I) \tag{2}$$